PROJECT EXPLANATION AND EXTERNAL VIVA PREPARATION GUIDE

A source-based guide to the Real Estate Management Platform

Prepared from the project files available in the workspace
Review date: 18 October 2026

What this guide is based on

The submitted ZIP was not present in the workspace when this analysis was performed. The guide therefore analyzes the available project source tree at D:\reactestate1 (React frontend and Spring Boot backend). It is grounded in the files present, not in an assumed ZIP. If the ZIP differs from this checkout, compare it before using any implementation detail in a viva.

Project identity used here: Real Estate Management Platform (inferred from backend metadata and feature set). A formal college project title, student details, institution, and approved problem statement were not found in the available files.

Contents

Use the page numbers added by the PDF viewer. Sections are arranged to support both reading and rapid viva revision.

01. Evidence boundary and project snapshot
02. Project overview and requirements
03. Architecture and end-to-end workflow
04. Technology stack
05. Source-code map and key logic
06. Database model and API guide
07. Installation and local run
08. Practical viva demonstration script
09. 2-minute and 5-10-minute presentations
10. 50 viva questions with answers
11. Glossary and quick revision
12. Source files reviewed

1. Evidence boundary and project snapshot

Archive status: No ZIP archive was found under the project workspace, and GitHub repository search for “reactestate1” returned no matching installed repository. The available checkout is at D:\reactestate1. All descriptions below are based on this checkout.

Project title: “Real Estate Management Platform” is a descriptive title inferred from the Maven project description, application naming, domain model, and UI modules. The code does not include an official student project title page.

What can be verified: the source tree includes a TypeScript/React/Vite frontend; a Java 21, Spring Boot backend; PostgreSQL/JPA entities and Flyway migration files; REST controllers; JWT security; local/S3 document storage; WebSocket chat; payment, email and Firebase integrations; and a set of backend service tests and frontend verification scripts.

Important uncertainty: database migration V1 is explicitly a baseline placeholder that asks maintainers to replace it with the full schema. The entity classes reveal modeled tables and relationships, but a clean database cannot be completely recreated from V1 alone with certainty. `application.properties` uses Hibernate `ddl-auto=update` in development; production properties enable Flyway. Validate the live deployment and migration history before claiming a tested clean install.

Do not claim unverified items: do not state that every external service is configured or active, that payments complete against live Razorpay, or that the checked-in UI and production backend have matching versions. Environment examples leave external credentials blank.

2. Project overview

This is a multi-role web application for organizing real-estate projects and supporting property sales operations. The implementation combines property/project catalog management with staff accounts, leads and follow-ups, documents, sales, subscriptions/payments, location data, dashboards, and chat.

Problem statement

Property businesses often keep project descriptions, unit availability, inquiries, sales follow-up, documents, and staff activity in separate spreadsheets or informal message threads. That makes it harder to find current information, assign responsibility, and track an inquiry through to a sale. This application provides a shared role-aware system for those workflows. This is a problem statement inferred from the actual modules; no authored problem statement was included.

Purpose and objectives

- Maintain real-estate projects, configurations, towers/buildings, floors, units, location and media metadata.
- Support distinct Super Admin, Admin, Sub Admin and Employee experiences with role-aware access.
- Capture and assign leads, record statuses, plan follow-up actions and review sales analytics.
- Store project, unit, chat and KYC documents with selectable local or AWS S3 storage.
- Provide account authentication and profile, password recovery, subscription/payment and communication workflows.
- Offer a single browser interface, with Capacitor configuration indicating an Android packaging path.

Target users

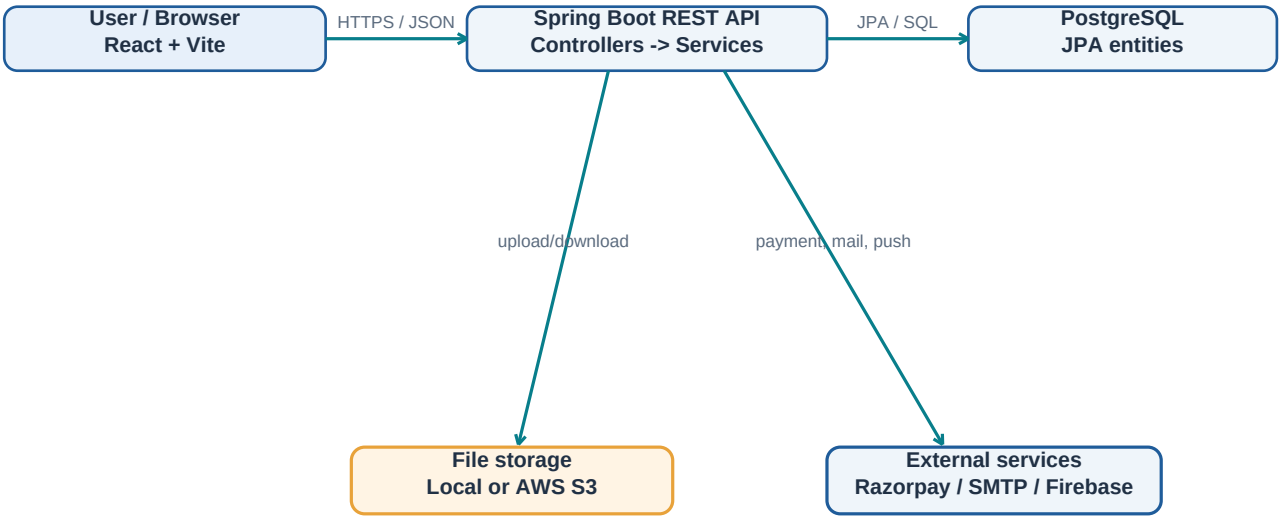
Role	Likely responsibilities supported by code
Super Admin	Platform-wide user, subscription, KYC and administration operations.
Admin	Manage sub-admins and employees, inspect operational data.
Sub Admin	Manage assigned/location-scoped property operations and teams.
Employee	Work on permitted properties, leads, follow-ups and sales.
Prospective/registered business users	Register, authenticate, view role-specific areas, submit/maintain information.

Key features visible in the source

- Role-based login and session restore; bearer JWT on API calls.
- Project creation, editing, search, sale/unit workflows, trash/restore, documents and taxonomy fields.
- Property hierarchy: project/property -> building -> floor -> unit, with unit documents.
- Lead assignment/status, CRM follow-ups, dashboard activity/statistics, sales recording and analytics.
- Locations, custom cities/areas and map URL handling.
- Chat conversations, message history/read state/reactions/edit/delete, file upload, presence, typing, call signaling and WebSocket endpoints.
- Subscription and Razorpay payment order/verification/webhook flows.
- OTP password recovery by email; optional Firebase push/custom token features.

3. Architecture and complete project flow

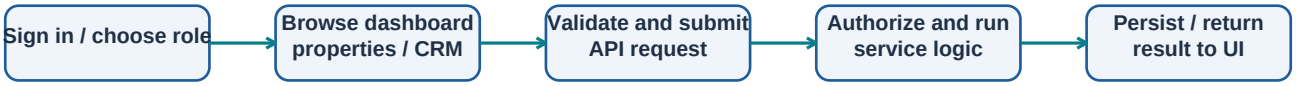
The application uses a client/server design. React manages the screens and interaction; the Java API validates and authorizes requests; services coordinate rules and repositories; JPA maps objects to PostgreSQL. Files may be stored separately from the relational rows.



WebSocket chat runs through the backend at /ws; frontend also supports Capacitor native packaging.

Figure 1. High-level system architecture. Optional service boxes are integrations present in source/configuration; a working external account and credentials are required for them to operate.

Startup and request flow



Errors are translated into API status/message and displayed in the page state.

- Frontend startup: Vite serves `REACT/index.html`; `src/main.tsx` creates the React root, installs QueryClientProvider, BrowserRouter and AuthProvider, then loads global design styles.
- App startup: `App.tsx` wraps the route app in motion preferences and Framer Motion policy, then renders `V0App`. `V0App.tsx` contains route resolution and application shell.
- Backend startup: `KunalApplication.main` calls SpringApplication.run. Spring discovers controllers, services, repositories and configuration; `@EnableScheduling` enables scheduled follow-up/cleanup jobs.
- A user action calls a feature API helper. `api/client.ts` adds JSON headers, optional bearer token, query strings, parses JSON/text, and raises ApiError for non-2xx responses.
- Spring Security applies request filters. JWT authentication resolves the user and authorities; authorization matchers and method annotations control access. The controller validates request shape and delegates business work to a service.
- The service uses repositories/JPA entities and transactions where configured. PostgreSQL stores normalized business data; storage service handles documents; integrations call external systems when configured.
- The response travels back as JSON (or binary file content), and React Query/UI state updates the screen. Chat may exchange live events through STOMP over WebSocket at `/ws`.

User journey (typical project listing)

Step	User action and system behavior
1. Sign in	Choose a role/login. AuthContext sends credentials to POST `/api/auth/login`, stores returned token, resolves the role, then loads `/api/auth/me` when restoring a session.
2. Enter workspace	Role-aware route and sidebar show permitted dashboards and modules. Query helpers fetch dashboard/project data.
3. Create project	User fills a property/project form; frontend validation and payload mapping submit to POST `/api/projects/create`.
4. Process	Controller delegates to ProjectService; service checks access/business rules and persists Project plus configurations through JPA.
5. Add media	Document endpoints accept multipart uploads and storage service writes locally or to S3; database keeps metadata/URLs.
6. Review/search	GET project/list/search endpoints return records. Frontend renders cards/details and may combine related data.
7. Convert/track	Lead, follow-up, unit, sale and payment modules handle downstream operational work.
8. Exit	Logout clears token/session state, query cache and chat sockets.

4. Technology stack

Technology	Role in this project
TypeScript 5.8 + JavaScript	Typed frontend logic and browser behavior; source has TS/TSX and some JSX.
React 19	Component-based user interface.
Vite 7	Frontend dev server and production bundler. `npm run dev` / `npm run build`.
React Router 7	Client-side navigation; BrowserRouter used at root.
TanStack React Query 5	Async server state, query cache and mutation support.
Spring Boot 4.0.3 / Java 21	Backend application, dependency injection, REST layer and runtime.
Spring MVC / Validation / Security	HTTP APIs, request validation, authentication and authorization.
Spring Data JPA + Hibernate	Repository persistence and mapping Java entities to relational tables.
PostgreSQL (+ H2 runtime dependency)	Configured default relational database; H2 dependency is present for alternate/testing contexts.
Flyway	Versioned SQL migration mechanism. Present in backend dependency and migration directory; baseline migration itself is a placeholder.
JWT (JJWT)	Signed bearer-token handling. JWT secret and expiry are externalized settings.
WebSocket / STOMP / SockJS	Chat events, notifications/signaling and browser transport.
AWS SDK S3	Optional remote object storage for documents; local filesystem is default in `application.properties`.
Razorpay Java SDK	Payment order and payment verification integration. Requires keys/secrets.
Spring Mail / Brevo SMTP	OTP/password recovery email delivery. Requires SMTP credentials.
Firebase Admin	Optional Firebase custom tokens and push notifications. Requires service account/project.
Capacitor 8	Native Android wrapper support and preferences storage.

Technology	Role in this project
Leaflet / React Leaflet	Map display and property location selection.
Radix UI, Tailwind CSS 4, Framer Motion, Recharts, Lucide	Accessible UI primitives, styling, animation, charts and icons.
Maven Wrapper / npm	Backend and frontend dependency/build tooling.

Architecture in plain language

The browser is the presentation layer: it gathers input and displays data. The backend is the rule and security layer: it checks the request, role and business constraints. PostgreSQL is the durable record store. File storage keeps large documents outside relational rows while their metadata remains addressable. Optional providers handle payment, email, maps and push delivery. The API boundary keeps the frontend from connecting directly to the database.

5. Source-code map and key logic

Path	Purpose / how to explain it
REACT/src/main.tsx	Frontend entry point. Creates root and mounts query, router and auth providers, then loads shared CSS.
REACT/src/App.tsx	App-level motion policy; renders V0App and conditionally decorates private routes.
REACT/src/v0/V0App.tsx	Main page/route composition; inspect this for route-to-page mapping.
REACT/src/auth/AuthContext.tsx	Session state, login/logout, role resolution, token restore and `/me` refresh.
REACT/src/api/client.ts	Shared HTTP helper; serializes JSON/FormData, attaches bearer token, parses response, raises ApiError.
REACT/src/api/estateApi.ts, featureApis.ts, mutations.ts	Feature-level endpoint wrappers and write operations.
REACT/src/config/env.ts	Same-origin defaults for web API/WebSocket; native platform defaults to deployed HTTPS/WSS URLs; optional VITE overrides.
REACT/src/services/tokenStore.ts, storage.ts	Token in-memory access and browser/native persistence abstraction.
REACT/src/types/domain.ts, api-contracts.ts	Shared domain and request/response types.
REACT/src/v0/app/[role]/**	Role-scoped screens: projects, properties, CRM, chat, map, analytics, sales, users, locations, subscription.
REACT/src/v0/components/**	Shared shell, forms, dialogs, property components, UI primitives and effects.
backend/.../KunalApplication.java	Spring Boot entry point; scheduling enabled.
backend/.../Controller/**	HTTP endpoints. Controller delegates to services instead of containing all domain logic.
backend/.../Service/**	Authentication, project/property, lead/CRM, sales, storage, chat, KYC, payment and other business services.
backend/.../Repository/**	Spring Data persistence interfaces.
backend/.../Entity/**	JPA entity model for accounts, projects, leads, follow-ups, property units, chat, sales, payments and locations.
backend/.../Security/** and Config/**	JWT/rate-limit filters, security chain, CORS and integration configuration.
backend/src/main/resources/application*.properties	Environment-driven runtime, database, upload, storage, JWT, mail, actuator and production configuration.

Path	Purpose / how to explain it
backend/src/main/resources/db/migration/**	Flyway SQL change files V1-V18.
REACT/package.json / package-lock.json	Frontend dependencies and runnable scripts with a lockfile.
backend/pom.xml / mvnw(.cmd)	Maven dependencies, Java version and wrapper commands.
backend/src/test/**, REACT/scripts/**, REACT/tests/**	Backend service tests and frontend contract/parity/readiness checks.

Authentication request logic (logical explanation)

1) Login page submits email/password and selected role through the estateApi wrapper. 2) Backend `/api/auth/login` delegates to AuthService and returns a token plus user/role data. 3) The client extracts token or jwt, writes it through tokenStore/storage, derives a normalized role from response/user/JWT, and updates React state. 4) Later API calls attach `Authorization: Bearer `. 5) On startup the client reads stored token and calls `/api/auth/me`; if backend is unavailable, it has a JWT-payload fallback for a provisional UI session. That fallback does not prove the token is accepted by the server.

Project persistence logic (logical explanation)

The `Project` entity maps to `projects`. It has scalar project/property metadata, a location reference, timestamps, a soft-delete flag, and a one-to-many eager list of `ProjectConfig` children with cascade/orphan removal. Helpers calculate total units and sold units from configs and calculate the lowest config price. The controller exposes create/list/detail/update/search/trash/restore/sell flows and delegates to ProjectService. Document metadata is handled through separate upload/download endpoints.

Code snippet: shared API helper

```
const token = tokenStore.get();
if (options.body !== undefined) {
  headers.set('Content-Type', 'application/json');
}
if (token) headers.set('Authorization', `Bearer ${token}`);
const response = await fetch(joinUrl(env.apiBaseUrl, path), { method, headers, body });
if (!response.ok) throw new ApiError(response.status, message, payload);
```

Explain it as: retrieve the current token; encode JSON requests; send the request to the configured API base; parse the response; turn HTTP errors into one consistent exception type. File uploads pass FormData and avoid manually setting the multipart boundary.

6. Database and API guide

Database approach

The application uses PostgreSQL by default, with Hibernate/JPA entity mappings. Data is split into entity groups. Foreign keys are visible in later migration scripts. Migrations V1-V18 progressively change the schema, but V1 is not a complete schema baseline. Development configuration defaults Hibernate DDL mode to `update`; production config enables Flyway. Confirm actual production schema and Flyway history before asserting exact column constraints.

Entity/table group	Role and key relationships
users, roles / temp_registrations, otp_records	Accounts, roles, pending registration and password recovery. Users reference optional custom city and area; employees connect to user accounts in a migration.
projects, project_configs, project_documents	Project catalog, unit configurations and media/docs; projects can reference locations and have soft-delete fields.
buildings, floors, property_units, unit_documents, project_units	Hierarchical inventory. Project -> Building -> Floor -> Unit, with project/building/floor foreign keys and cascading deletes in V16 migration.

Entity/table group	Role and key relationships
locations, custom_cities, areas, broker_locations	Geographic records and subscription rates/active state; leads and users may reference area/city.
leads, follow_ups, activities, sales	Lead pipeline, owner/creator-assigned follow-up, user activity and sale transaction records. Leads can reference assigned user and area.
payments, plans, subscriptions	Subscription plan and payment lifecycle; payment provider identifiers stored to correlate verification.
conversations, conversation_participants, messages, message_reads, message_reactions	Chat room participation, messages, read markers and reactions.
user_kyc_documents	KYC document metadata linked to users or temporary registration flows.

CRUD and data lifecycle

- Create: POST endpoint receives a JSON DTO or multipart file; service validates and creates entity/metadata.
- Read: GET endpoint loads by ID, list, filter, search or parent relationship; entities/DTOs are serialized to JSON.
- Update: PUT/PATCH updates selected fields after authorization and ownership/scope checks.
- Delete: DELETE may remove a record, while project “trash” endpoints implement soft deletion and restore. Child relationships may use database/JPA cascade behavior; do not assume every DELETE is permanent.
- Files: binary contents go to local uploads or S3 according to storage configuration; database rows keep names, ownership, type and/or URL metadata. Deletion should remove metadata and storage object as service logic dictates.

API map (representative routes)

Feature	Example method + route	Purpose
Authentication	POST `/api/auth/login`; GET `/api/auth/me`	Create session and retrieve current account.
Projects	POST `/api/projects/create`; GET `/api/projects`; GET `/api/projects/{id}`	Create/list/read projects. Other routes include search, update, trash/restore, sell, and cover photo.
Property hierarchy	GET/POST `/api/properties/{propertyId}/buildings`; nested floors and units	Maintain building/floor/unit inventory.
Leads	POST `/api/leads/create`; POST `/api/leads/assign`; POST `/api/leads/update-status`; GET `/api/leads/all`	Capture, assign, progress and list leads.
CRM follow-up	POST `/api/crm/leads/{leadId}/follow-ups`; PATCH `/api/crm/follow-ups/{id}/complete`	Schedule and complete follow-ups.
Documents	POST `/api/documents/upload/{projectId}`; GET `/api/documents/{projectId}`; DELETE `/api/documents/{documentId}`	Upload, list and delete project documents. Multipart uploads have configured size caps.
Sales	GET `/api/sales`; POST `/api/sales/record`; GET `/api/sales/analytics`	Record sales and inspect analysis.
Locations	GET/POST `/api/locations`; GET/POST `/api/areas`	Manage location data; write access is role-restricted.
Payments	POST `/api/payment/create-order`; POST `/api/payment/verify`; POST `/api/payments/webhook`	Start and verify a Razorpay payment; webhook handles provider callbacks.
Chat REST + live	GET `/api/chat/conversations`; WebSocket `/ws` (STOMP mappings such as `/chat.send`)	Load chat data and exchange live messages/events.

Request/response contract

Most endpoints exchange JSON; upload endpoints use multipart form data; download endpoints may return a binary response. The shared frontend client parses JSON based on Content-Type or falls back to text. Errors become an ApiError carrying HTTP status, message and details. The full set of supported routes is listed in `REACT/src/api/endpointCatalog.ts`; confirm exact payload DTO fields in each controller/DTO before giving a precise example.

7. Install and run locally

Prerequisites

- Node.js and npm compatible with the Vite 7 project (use a current LTS release).
- Java 21 JDK (backend `pom.xml` sets java.version 21).
- PostgreSQL, with a database named `realestate` and a local user/password.
- Internet access for Maven/npm dependencies on the first install. Optional provider credentials only for features that need them.

Configure database

Create a PostgreSQL database named `realestate`, then supply DB_URL, DB_USERNAME and DB_PASSWORD as environment variables. Avoid relying on credentials written in development properties. Backend env.example describes the expected values. For local development, set a strong JWT_SECRET (at least 32 characters per env.example); optional mail, Firebase, Razorpay and S3 values can remain unset if those features are not being demonstrated.

Commands (PowerShell, from project root)

```
# Terminal A: backend
cd backend
$env:DB_URL='jdbc:postgresql://localhost:5432/realestate'
$env:DB_USERNAME='postgres'
$env:DB_PASSWORD='YOUR_LOCAL_DB_PASSWORD'
$env:JWT_SECRET='replace-with-a-random-secret-at-least-32-chars'
.\mvnw.cmd spring-boot:run

# Terminal B: frontend (open a second PowerShell at project root)
cd REACT
npm ci
npm run dev
```

The backend listens on port 8080 by default. The Vite app runs at the URL shown by Vite (configured for 127.0.0.1). Browser `env.ts` defaults API traffic to same-origin `/api`; a local dev proxy must route `/api` and `/ws` to the backend if using this setup. Inspect `vite.config.ts` before demonstration. Set VITE_API_BASE_URL and VITE_WS_URL if needed to point the browser directly to local backend ports.

Build/verification commands

```
# frontend production build
cd REACT
npm run build

# backend tests
cd ..\backend
.\mvnw.cmd test
```

These are the project's available build/test commands; this guide does not claim they were run. For a viva, first ensure database migrations/schema exist and seed/prepare a demo user using an authorized registration/admin path. No seed SQL or known demo credentials were found in the available project files. Do not use example test credentials unless you have verified they exist in your own local database.

8. Practical demonstration script

Run the application and verify your demo account, database and integrations before the exam. Use a prepared sample project and non-sensitive sample data.

Time	What to show	What to say
0:00-0:30	Landing/role selection and login page.	“This is a real-estate operations platform. It separates workflows by user role. I will sign in with a prepared local demo account.”
0:30-1:15	Role dashboard and sidebar.	“The application restores the authenticated user and role, then loads role-specific dashboard information through backend APIs.”
1:15-2:15	Project list, search and a project detail.	“Projects store core location, builder and property type information, while configurations represent available unit types and pricing.”
2:15-3:15	Create/edit a sample project or show existing project wizard.	“The UI submits validated data to the project API. Spring controller delegates to the service; JPA persists the project and configurations in PostgreSQL.”
3:15-4:00	Project documents / cover image if storage is configured.	“The document content is stored using the configured storage provider, while the backend tracks metadata and access.”
4:00-5:00	Lead list, assign lead, update status, create/complete follow-up.	“A lead is assigned to a user and progresses through status updates; follow-ups help staff record the next contact action.”
5:00-5:45	Sales/analytics screen.	“Sales records and dashboard APIs summarize operational activity for permitted roles.”
5:45-6:30	Optional chat or location map.	“Chat uses REST for history and WebSocket/STOMP for live events. Location data and map components support geographic context.”
6:30-7:00	Logout and quick architecture recap.	“The browser communicates with the API, the API enforces security and business rules, and PostgreSQL stores structured data.”

If the examiner asks about unavailable services, say: “The integration code is present, but this local demo environment does not have the external provider credentials configured, so I will explain its flow from source rather than claim a live transaction.”

9. Presentation and explanation preparation

2-minute introduction

“Good morning. My project is a Real Estate Management Platform for organizing property listings and the day-to-day work around them. The system supports different roles such as Super Admin, Admin, Sub Admin and Employee. Users can manage real-estate projects and unit configurations, organize buildings and units, handle leads and follow-ups, record sales, manage documents and communicate through chat. The frontend is built with React and TypeScript using Vite. It communicates with a Java 21 Spring Boot backend through REST APIs. Spring Security and JWT provide authenticated, role-based access. PostgreSQL stores the structured data through JPA entities, while document files can use local storage or AWS S3. Optional integrations cover Razorpay payments, SMTP email and Firebase notifications. A typical flow is: the user signs in, the frontend sends an authorized API request, the backend checks permissions and business logic, stores or reads data, and sends the result back to the screen. The main value is having property, inquiry, follow-up and sales information in one role-aware system. The available project files do not contain a formal college problem statement or a complete fresh-install baseline migration, so I describe those implementation boundaries accurately.”

Detailed 5-10 minute explanation

Minute 0-1: Need and users. Explain the operational problem: fragmented property data and sales follow-up. Introduce the role groups and say the application centralizes workflows rather than acting only as a public property website.

Minute 1-2: Frontend. Point out `main.tsx`: React root, router, query cache and auth provider. Explain that the role-based app renders pages for projects/properties, CRM, chat, analytics, sales, locations and subscriptions. React Query handles server data, while shared API wrappers attach auth tokens.

Minute 2-3: Backend and security. Show `KunalApplication.java` as the entry point. Explain Controller -> Service -> Repository -> Entity flow. Spring Security filters validate JWT; role authorities restrict endpoints, and BCrypt is used as the password encoder.

Minute 3-4: Data model. Show the Project entity and explain ProjectConfig child records, then the project/building/floor/unit hierarchy. Relate leads to follow-ups and users. Mention that V1 migration is a placeholder and production Flyway history should be inspected to reproduce a database.

Minute 4-5: Demonstrate a workflow. Sign in, open project list/detail, then leads and follow-ups. If available, create or edit a project using sample data. Explain that user input becomes an HTTP request, is validated, saved by the service and returned as JSON.

Minute 5-6: Files, chat and integrations. Demonstrate a document or chat if configured. Explain that document bytes use local/S3 storage and metadata stays in the database. WebSocket/STOMP supports real-time chat; Razorpay, SMTP and Firebase require external configuration.

Minute 6-7: Testing, limitations and next steps. Point to backend service tests and frontend scripts. Do not claim tests passed unless you ran them. Mention fresh-install schema gap, credentials/configuration dependence, and possible improvements: stronger migration baseline, end-to-end tests, audit trail, broader input validation/monitoring and backup/restore procedures.

Simple beginning-to-end answer

“The user opens the React interface and signs in. The frontend sends the credentials to the Spring Boot API. The backend authenticates the account and returns a JWT. The frontend stores the session token and uses it on later requests. When the user creates or views a project, the API checks the token and role, applies service rules and reads or saves entities through JPA in PostgreSQL. If the user uploads a document, the file is written to local or S3 storage and its metadata is saved. The backend returns a response, and the frontend updates the screen. The same pattern supports leads, follow-ups, property units, sales and payments, with WebSockets used for real-time chat events.”

10. Viva questions and answers

These answers describe the implementation visible in the source. Adapt the user story to the features you personally implemented and can demonstrate.

1. What is the project?

A role-aware real-estate management platform for projects, inventory, leads, follow-ups, sales, documents and team workflows.

2. Why did you choose this project?

It brings common property-business operations into a single system and gives practical experience with frontend, APIs, security and persistence.

3. What problem does it solve?

It reduces fragmentation in property/project information and inquiry follow-up by making those records accessible through one application.

4. Who are the users?

The code models Super Admin, Admin, Sub Admin and Employee roles; exact permissions are enforced in backend security and method checks.

5. What are the main modules?

Authentication, projects/properties, locations, leads/CRM, sales, documents, chat, users/admin, subscriptions and payments.

6. What is the frontend stack?

React 19, TypeScript, Vite, React Router, TanStack React Query, Tailwind CSS and Radix-based UI components, among other listed libraries.

7. What is the backend stack?

Java 21 and Spring Boot 4 with Spring MVC, Security, Validation, Data JPA, Hibernate and integrations.

8. Which database is used?

PostgreSQL is the configured default. H2 is also included as a runtime dependency for alternate contexts.

9. What is the frontend entry point?

`REACT/src/main.tsx`; it mounts React with router, query and authentication providers.

10. What is the backend entry point?

`KunalApplication.java`; its main method invokes SpringApplication.run and scheduling is enabled.

11. What is a REST API?

A web interface where clients use HTTP methods and resource paths to exchange data, usually JSON.

12. What does a controller do?

It maps an HTTP route to a Java method, accepts input, and delegates the work to a service.

13. What does a service do?

It contains business operations and coordinates persistence or integrations.

14. What does a repository do?

It provides the data-access methods used to query and save JPA entities.

15. What is JPA?

Java Persistence API is the standard object-relational mapping interface; Hibernate is the provider used here.

16. What is an entity?

A Java class mapped to a relational database table, such as Project mapped to `projects`.

17. How does a project relate to configurations?

Project has a one-to-many list of ProjectConfig children with cascade and orphan removal configured.

18. Explain the inventory hierarchy.

A project can have buildings; buildings can have floors; floors can have property units. The V16 migration defines foreign keys and cascading behavior.

19. What is a foreign key?

A database constraint that links a row to a valid row in another table.

20. What is a migration?

A versioned schema change script used to evolve database structure in a controlled sequence.

21. Can this ZIP initialize a fresh database?

The archive was not available; in the local tree V1 is explicitly a baseline placeholder, so it does not prove a complete clean schema setup.

22. What does Hibernate ddl-auto update do?

It asks Hibernate to adjust schema based on entities at startup. It can be useful locally but is less controlled than reviewed production migrations.

23. What is JWT?

A signed token that carries claims; the backend validates it and uses its identity/role to authorize requests.

24. Where does the client add the JWT?

The shared API client adds `Authorization: Bearer ...` when a token is present.

25. How are roles checked?

Spring Security route matchers and method-level security constrain endpoints to specified roles.

26. How are passwords encoded?

SecurityConfig provides BCryptPasswordEncoder. Never store plain text passwords.

27. What is CORS?

A browser policy controlling cross-origin web requests. This backend has a dedicated CORS configuration and allowed-origin setting.

28. How is login protected from brute force?

A RateLimitFilter is wired into the security chain, with login rate-limit settings in application properties.

29. How does the frontend handle errors?

It parses the response and throws ApiError with HTTP status, message and details for non-success responses.

30. What is React Query used for?

Managing asynchronous server data, caching queries and coordinating updates after mutations.

31. How are files stored?

The storage configuration supports local filesystem storage by default and AWS S3 in production configuration.

32. Are file contents stored in PostgreSQL?

The intended design stores file bytes in local/S3 object storage and records metadata or URLs in the relational database.

33. How does chat work?

REST endpoints manage conversation history and actions; WebSocket/STOMP endpoints handle live events like sending, typing and read state.

34. What is STOMP?

A messaging protocol layered over WebSocket that defines destinations and message frames.

35. How are leads and follow-ups connected?

A follow-up references a lead and has creator/owner user references, supporting assigned next-contact actions.

36. How are project deletes handled?

Project routes include trash/restore and permanent delete, and the Project entity has soft-delete fields.

37. What is soft deletion?

Marking a record deleted without immediately removing the row, allowing restore or retention.

38. How are payments integrated?

Razorpay order creation, verification, renewal and webhook endpoints are present; real use requires configured credentials and provider setup.

39. How is password recovery implemented?

Auth endpoints cover sending an OTP, verifying it and resetting the password; mail configuration uses SMTP settings.

40. What is Firebase used for?

Source includes Firebase Admin services for push/custom-token support; credentials are optional environment settings.

41. How does the application start locally?

Start PostgreSQL, set backend environment variables and run Maven wrapper `spring-boot:run`; then run `npm ci` and `npm run dev` in REACT.

42. What is Vite?

A frontend development server and bundler that serves the app during development and creates a production build.

43. What is TypeScript?

JavaScript with static type checking, helping detect many contract and value-shape mistakes before runtime.

44. What are the benefits of a layered architecture?

Separating routes, business rules and data access makes responsibilities easier to understand and maintain.

45. What tests are present?

Several Java service tests and frontend scripts/tests for API contracts, parity, motion and readiness are present; results must be run locally before claimed.

46. What security controls are visible?

JWT authentication, role restrictions, BCrypt password encoding, login rate limiting, allowed-origin configuration, secure headers and externalized secrets.

47. What is a limitation?

The checked-in V1 SQL migration is a placeholder; reproducing a clean database needs a verified baseline or existing database schema.

48. What happens if Razorpay credentials are missing?

Payment integration cannot complete real provider operations; the code/config alone is not proof of a live payment setup.

49. What challenge did you face?

A defensible project-specific answer is integrating several modules with shared role checks and keeping frontend API contracts aligned. Only claim challenges you personally experienced.

50. What did you learn?

The code supports a learning narrative about React composition, typed API clients, Spring layers, entity modeling, JWT security and external-service configuration.

51. What would you improve next?

Add a full reviewed baseline migration, end-to-end tests, audit logging, richer observability, consistent API docs and documented demo seeding.

52. How would you scale it?

Use stateless backend replicas, managed PostgreSQL with indexes/backups, object storage/CDN for files, queues for slow integrations and monitoring. These are proposals, not current guarantees.

53. How do you protect uploaded documents?

The code has authenticated document/KYC routes and configurable storage, but access policy and bucket permissions must be audited per deployment. Avoid claiming every file is private without verifying.

54. Why not connect frontend directly to database?

The API centralizes authorization, validation and business logic and prevents exposing database credentials to browsers.

55. How do you explain the full flow quickly?

The user acts in React; the client calls a secured Spring endpoint; the service applies rules and persists through JPA to PostgreSQL; the response updates the UI.

11. Technical glossary and quick revision

Term	Simple meaning
API	A defined way for software parts to request data or actions from one another.
JWT	A signed token used to carry identity/role claims between client and server.
ORM	Mapping between program objects and relational database tables.
Entity	A class representing a database record type.
Foreign key	A link enforcing a relationship between tables.
CRUD	Create, Read, Update and Delete operations.
DTO	A request/response data shape used at the API boundary.
CORS	Browser rules for requests made across different website origins.
WebSocket	A persistent two-way connection useful for live updates.
STOMP	A messaging format/protocol commonly used over WebSocket.
Migration	A recorded change to a database schema.
Soft delete	Mark a record deleted so it can be restored or retained.
Idempotent	Repeating an operation produces the same effect; often important for safe retries.
Object storage	A service for files/blobs, separate from relational database rows.
Role-based access	Allowing or denying actions based on a user role.

30-second quick revision

- Project: Real Estate Management Platform (descriptive inferred title).
- Frontend: React + TypeScript + Vite. Entry: `REACT/src/main.tsx`.
- Backend: Java 21 + Spring Boot. Entry: `KunalApplication.java`.
- Database: PostgreSQL with JPA/Hibernate; migrations V1-V18, but V1 is a placeholder.
- Security: JWT bearer auth, role checks, BCrypt password encoder and rate-limit filter.
- Core flows: projects/configurations/property units; leads/follow-ups/sales; documents; chat; locations; subscriptions/payments.
- Local demo: PostgreSQL + backend Maven wrapper + React Vite dev server; verify Vite API proxy/environment first.
- Truthful caveat: external credentials and fresh DB schema are not proven by source alone.

12. Source files reviewed

The analysis sampled the project manifests, configuration, core entry points, entity mappings, controllers, migrations, endpoint catalog, API client, authentication context and documentation. Key paths:

- `REACT/package.json`, `REACT/package-lock.json`, `REACT/vite.config.ts`, `REACT/src/main.tsx`, `REACT/src/App.tsx`, `REACT/src/v0/V0App.tsx`.
- `REACT/src/auth/AuthContext.tsx`, `REACT/src/api/client.ts`, `REACT/src/api/endpointCatalog.ts`, `REACT/src/config/env.ts`.
- `backend/pom.xml`, `backend/env.example`, `backend/src/main/resources/application.properties`, `application-prod.properties`.
- `backend/src/main/java/com/real_estate/Kunal/KunalApplication.java`, Controller, Service, Repository, Entity, Security and Config packages.

- `backend/src/main/resources/db/migration/V1__baseline.sql` through `V18__property_type_taxonomy_and_towers.sql`.
- `backend/src/test/**`, `REACT/tests/**`, `REACT/scripts/**`, and root `TODO.md`.

Before the viva, compare this guide against the exact archive your college project was submitted from. The source reviewed here has no formal report, ERD, seed database dump or complete baseline migration, so use your project supervisor's approved title and requirements if they differ.